

INTRODUCCIÓN

En esta actividad, se aplican diversas consultas SQL sobre un conjunto de datos para simular la generación de informes que serían requeridos por una empresa real. Se busca demostrar el dominio de SQL básico y avanzado que permita filtrar, relacionar y analizar datos con precisión.



WHERE

¿Qué hace?

La cláusula WHERE permite filtrar registros de una tabla que cumplan una condición específica.

Ejemplo

```
SELECT ProductName,  
UnitPrice, CategoryID  
FROM Products  
WHERE CategoryID = 1;
```

Explicación

Se filtran los productos donde CategoryID = 1, que en Northwind corresponde a "Beverages".

EJEMPLO DESARROLLADO

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane shows the 'Northwind' database selected under 'Databases'. The main window shows a SQL query executed successfully, with the results displayed in a table. The query is: `SELECT ProductName, UnitPrice, CategoryID FROM Products WHERE CategoryID = 1;`

	ProductName	UnitPrice	CategoryID
1	Chai	18.00	1
2	Chang	19.00	1
3	Guaraná Fantástica	4.50	1
4	Sasquatch Ale	14.00	1
5	Steele's Stout	18.00	1
6	Côte de Blaye	263.50	1
7	Charlesheise white	18.00	1
8	Ipswich Coffee	46.00	1
9	Laughing Lumberjack Lager	14.00	1
10	Outback Lager	15.00	1
11	Rhinobrau Klosterbier	7.75	1
12	Lakkakoon	18.00	1

Query executed successfully. DESKTOP-JD46SLB\SQLEXPRESS... DESKTOP-JD46SLB\HP [66] Northwind 00:00:02 12 rows

LIKE

¿Qué hace?

LIKE permite buscar valores que coincidan parcialmente con un texto, usando comodines como:

- % (cualquier número de caracteres)
- _ (un solo carácter)

Ejemplo

```
SELECT ProductName,
       UnitPrice
FROM Products
WHERE ProductName LIKE
       'C%';
```

Explicación

'C%' significa "cualquier cadena que comience con la letra C". El símbolo % representa cualquier secuencia de caracteres después de la C.

EJEMPLO DESARROLLADO

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the 'Northwind' database. The main window shows a query editor with the following SQL code:

```
GO
SELECT ProductName, UnitPrice
FROM Products
WHERE ProductName LIKE 'C%';
```

Below the query editor, the Results pane displays the output of the query as a table with two columns: ProductName and UnitPrice. The results are as follows:

ProductID	ProductName	UnitPrice
1	Chai	18.00
2	Chang	19.00
3	Chef Anton's Cajun Seasoning	22.00
4	Chef Anton's Gumbo Mix	21.35
5	Camarones Tigers	62.50
6	Côte de Blaye	263.50
7	Chartreuse verte	18.00
8	Chocolade	12.75
9	Camembert Pierrot	34.00

The status bar at the bottom indicates that the query was executed successfully, returning 9 rows.

BETWEEN

¿Qué hace?

BETWEEN se usa para filtrar valores que estén dentro de un rango definido, como precios, fechas o números.

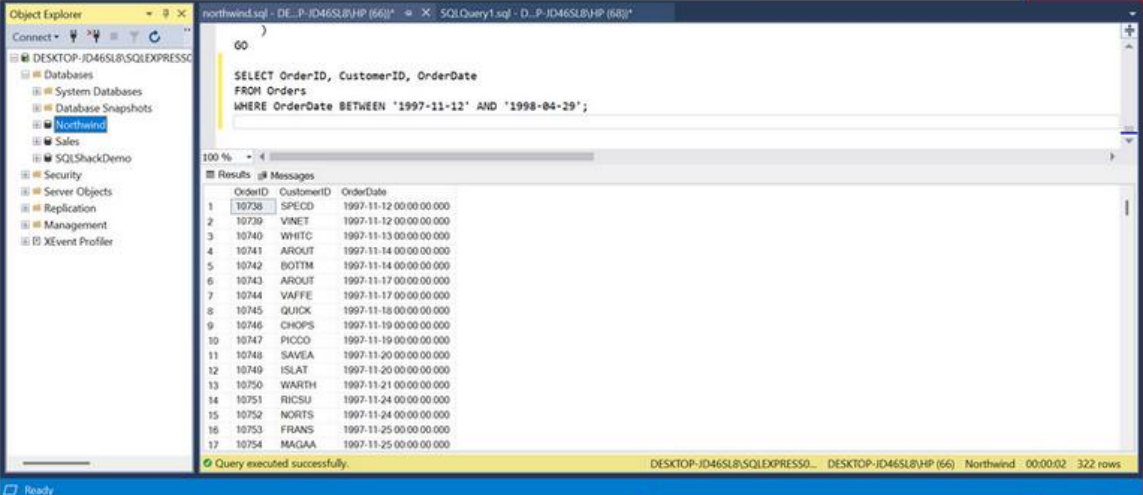
Ejemplo

```
SELECT OrderID, CustomerID,
       OrderDate
FROM Orders
WHERE OrderDate BETWEEN
'1997-11-12' AND '1998-04-29';
```

Explicación

OrderDate es el campo que se filtra. '1997-11-12' AND '1997-04-29' define el rango de fechas. La cláusula BETWEEN incluye ambos extremos del rango (rango de las fechas).

EJEMPLO DESARROLLADO



The screenshot displays the SQL Server Enterprise Manager interface. On the left, the Object Explorer shows the database structure, with the 'Northwind' database selected. The main window shows a SQL query executed successfully. The query is as follows:

```
SELECT OrderID, CustomerID, OrderDate
FROM Orders
WHERE OrderDate BETWEEN '1997-11-12' AND '1998-04-29';
```

The results are displayed in a table with 17 rows and 3 columns: OrderID, CustomerID, and OrderDate. The data is as follows:

OrderID	CustomerID	OrderDate
10738	SPECD	1997-11-12 00:00:00.000
10739	VINET	1997-11-12 00:00:00.000
10740	WHITC	1997-11-13 00:00:00.000
10741	AROUT	1997-11-14 00:00:00.000
10742	BOTTM	1997-11-14 00:00:00.000
10743	AROUT	1997-11-17 00:00:00.000
10744	VAFFE	1997-11-17 00:00:00.000
10745	QUICK	1997-11-18 00:00:00.000
10746	CHOPS	1997-11-19 00:00:00.000
10747	PICCO	1997-11-19 00:00:00.000
10748	SAVEA	1997-11-20 00:00:00.000
10749	ISLAT	1997-11-20 00:00:00.000
10750	WARTH	1997-11-21 00:00:00.000
10751	RICSU	1997-11-24 00:00:00.000
10752	NORTS	1997-11-24 00:00:00.000
10753	FRANS	1997-11-25 00:00:00.000
10754	MAGAA	1997-11-25 00:00:00.000

The status bar at the bottom indicates that the query was executed successfully, showing the number of rows (322) and the time taken (00:00:02).

JOIN

¿Qué hace?

JOIN permite relacionar datos de dos o más tablas usando una columna en común.

- INNER JOIN: solo muestra coincidencias en ambas tablas.
- LEFT JOIN: muestra todo de la tabla izquierda y coincidencias de la derecha.
- RIGHT JOIN: al revés del LEFT.

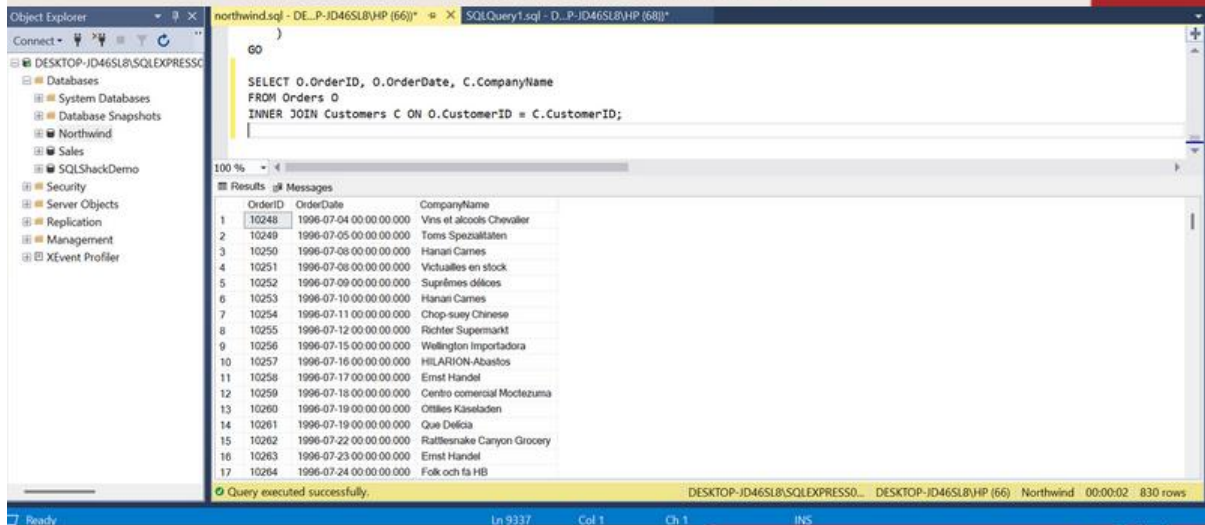
Ejemplo

```
SELECT O.OrderID,  
       O.OrderDate,  
       C.CompanyName  
FROM Orders O  
INNER JOIN Customers C ON  
O.CustomerID = C.CustomerID;
```

Explicación

Se utiliza INNER JOIN para combinar la tabla de pedidos (Orders) con la de clientes (Customers). La unión se realiza a través del campo CustomerID, que es común en ambas tablas. El resultado muestra el ID del pedido, la fecha del pedido y el nombre de la empresa.

EJEMPLO DESARROLLADO



The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Object Explorer' with the 'Northwind' database selected. The right pane shows a query window with the following SQL code:

```
GO  
  
SELECT O.OrderID, O.OrderDate, C.CompanyName  
FROM Orders O  
INNER JOIN Customers C ON O.CustomerID = C.CustomerID;  
  
GO
```

Below the query window, the 'Results' pane displays the output of the query as a table with 17 rows and 3 columns: OrderID, OrderDate, and CompanyName. The status bar at the bottom indicates 'Query executed successfully.' and '830 rows'.

OrderID	OrderDate	CompanyName
10248	1996-07-04 00:00:00.000	Vins et alcools Chevalier
10249	1996-07-05 00:00:00.000	Toms Spezialitäten
10250	1996-07-08 00:00:00.000	Hanari Carnes
10251	1996-07-08 00:00:00.000	Victualies en stock
10252	1996-07-09 00:00:00.000	Suprêmes délices
10253	1996-07-10 00:00:00.000	Hanari Carnes
10254	1996-07-11 00:00:00.000	Chop-suey Chinese
10255	1996-07-12 00:00:00.000	Richter Supermarkt
10256	1996-07-15 00:00:00.000	Wellington Importadora
10257	1996-07-16 00:00:00.000	HILARION-Abastos
10258	1996-07-17 00:00:00.000	Ernst Handel
10259	1996-07-18 00:00:00.000	Centro comercial Moctezuma
10260	1996-07-19 00:00:00.000	Ottiles Käseleraden
10261	1996-07-19 00:00:00.000	Que Pasa
10262	1996-07-22 00:00:00.000	Rattlesnake Canyon Grocery
10263	1996-07-23 00:00:00.000	Ernst Handel
10264	1996-07-24 00:00:00.000	Folk och fa H&B

GROUP BY

¿Qué hace?

GROUP BY agrupa los registros que tienen el mismo valor en una o más columnas para usar funciones agregadas como COUNT(), SUM(), AVG().

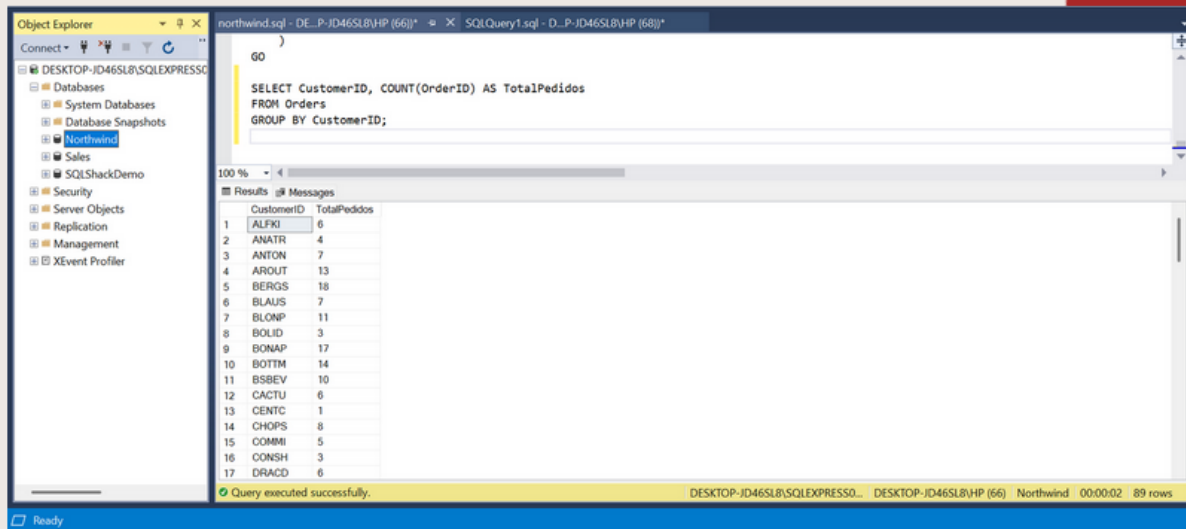
Ejemplo

```
SELECT CustomerID,  
COUNT(OrderID) AS  
TotalPedidos  
FROM Orders  
GROUP BY CustomerID;
```

Explicación

Agrupa los registros por cliente (CustomerID). Usa COUNT() para calcular cuántos pedidos ha hecho cada uno. Muestra todos los clientes, incluso si solo hicieron un pedido.

EJEMPLO DESARROLLADO



The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the database structure, including the 'Northwind' database. The main window shows a SQL query in the 'SQLQuery1.sql' file:

```
GO

SELECT CustomerID, COUNT(OrderID) AS TotalPedidos
FROM Orders
GROUP BY CustomerID;
```

Below the query, the 'Results' pane displays the output of the query, showing a list of customers and their total orders:

CustomerID	TotalPedidos
1 ALFKI	6
2 ANATR	4
3 ANTON	7
4 AROUT	13
5 BERGS	18
6 BLAUS	7
7 BOLID	3
8 BONAP	17
9 BOTTM	14
10 BSBEV	10
11 CACTU	6
12 CENTC	1
13 CHOPS	8
14 COMMI	5
15 CONSH	3
16 DRACD	6

The status bar at the bottom indicates 'Query executed successfully.' and '89 rows'.

HAVING

¿Qué hace?

HAVING se usa junto a GROUP BY para filtrar los resultados agregados.

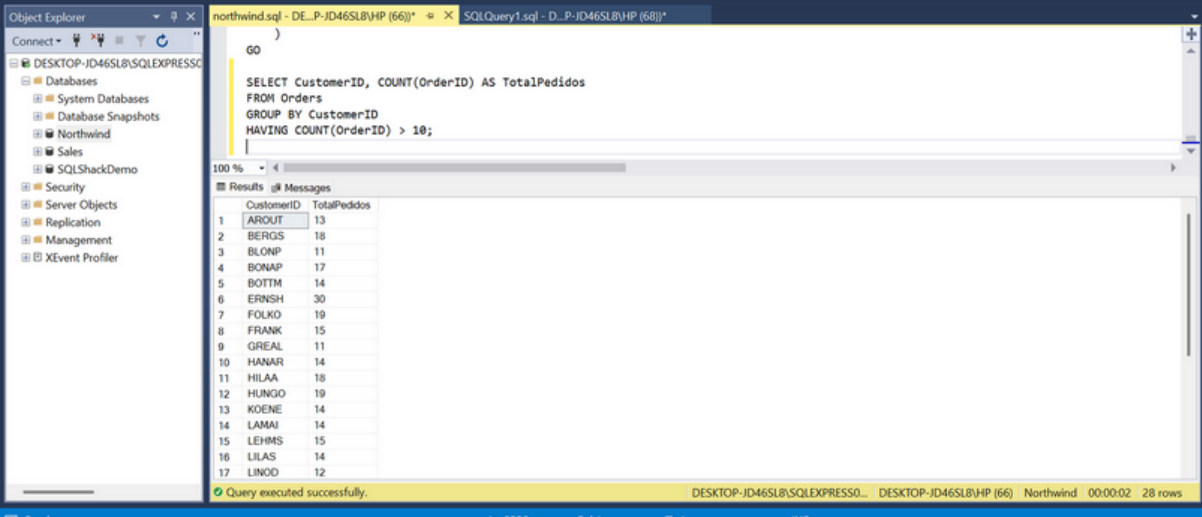
Ejemplo

```
SELECT CustomerID,
COUNT(OrderID) AS
TotalPedidos
FROM Orders
GROUP BY CustomerID
HAVING COUNT(OrderID) > 10;
```

Explicación

Igual que el ejemplo anterior, agrupa por CustomerID. Pero ahora, con HAVING, se filtran solo los grupos (clientes) cuya cantidad de pedidos sea mayor a 10. HAVING siempre se usa después de GROUP BY.

EJEMPLO DESARROLLADO



The screenshot shows the Microsoft SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the server structure, including Databases, System Databases, Database Snapshots, Northwind, Sales, SQLShackDemo, Security, Server Objects, Replication, Management, and XEvent Profiler. The main window displays a SQL query in the 'SQLQuery1.sql' file. The query is as follows:

```
GO  
  
SELECT CustomerID, COUNT(OrderID) AS TotalPedidos  
FROM Orders  
GROUP BY CustomerID  
HAVING COUNT(OrderID) > 10;
```

Below the query editor, the 'Results' pane shows the output of the query. It displays a table with two columns: 'CustomerID' and 'TotalPedidos'. The table contains 17 rows of data, representing customers with more than 10 orders. The status bar at the bottom indicates 'Query executed successfully.' and '28 rows'.

CustomerID	TotalPedidos
AROUT	13
BERGS	18
BLONP	11
BONAP	17
BOTTM	14
ERNSH	30
FOLKO	19
FRANK	15
GREAL	11
HANAR	14
HILAR	18
HUNGO	19
KOENE	14
LAMAI	14
LEHMS	15
LILAS	14
LINOD	12

CONCLUSIÓN

Las consultas SQL permiten depurar, analizar y presentar datos de forma efectiva. Dominar comandos como JOIN, WHERE o GROUP BY es fundamental en cualquier entorno empresarial. El uso de filtros adecuados mejora la toma de decisiones al enfocarse solo en la información relevante.

Nueva consulta SQL:

```
SELECT CustomerID, COUNT(OrderID) AS TotalPedidos
FROM Orders
GROUP BY CustomerID;
```

100 %

Results Messages

	CustomerID	TotalPedidos
1	ALFKI	6
2	ANATR	4
3	ANTON	7
4	AROUT	13
5	BERGS	18
6	BLAUS	7
7	BLONP	11
8	BOLID	3
9	BONAP	17
10	BOTTM	14
11	BSBEV	10

Es para mostrar el número de pedidos por cliente.

```
SELECT CustomerID, COUNT(OrderID) AS TotalPedidos
FROM Orders
GROUP BY CustomerID;
```